GRADO EN INGENIERÍA INFORMÁTICA · 3.ER CURSO

SISTEMAS DISTRIBUIDOS
PRÁCTICA 2: gRPC EN .NET 10

Guía Integral de Principio a Fin: Diseño de IDL, Concurrencia Multihilo en Memoria y Arquitectura Distribuida con C#

Asignatura:	Sistemas Distribuidos / Computación Distribuida
Ámbito Académico:	Escuela Técnica Superior de Ingeniería Informática (España)
Plataforma & Lenguaje:	.NET 10.0 SDK & C# 13
Middleware / IPC:	gRPC (Google Remote Procedure Call) sobre HTTP/2 (h2c)
Formato de Contrato:	Protocol Buffers v3 (proto3 IDL)
Patrón Concurrente:	Productor-Consumidor con Búfer Acotado en Memoria
Sincronización:	SemaphoreSlim, ConcurrentQueue, ConcurrentDictionary, lock

Manual Oficial de Prácticas y Guía de Defensa de Examen

Documento estructurado con rigor teórico-práctico para dominar la implementación paso a paso y responder con excelencia en la evaluación oral.

📋 **Índice General de Contenidos**

1. Fundamentos Teóricos de Sistemas Distribuidos
2. Arquitectura Global del Sistema
3. Construcción del Proyecto Desde Cero (CLI)
4. El Contrato de Comunicación (IDL Protobuf)
5. Configuración de los Proyectos (.csproj)
6. El Servidor (Restaurante): Concurrencia y Estado
7. El Agente Consumidor (Repartidor): Worker Loop
8. La Aplicación Cliente (Cliente): Menú y Stubs
9. Guía de Ejecución y Casos de Prueba Límite
10. Banco de 10 Preguntas Típicas de Examen
11. Chuleta de Depuración y Resolución de Errores
12. Conclusión y Competencias del Ingeniero

1. Fundamentos Teóricos de Sistemas Distribuidos

Para superar con la máxima calificación este laboratorio en 3º de Grado en Ingeniería Informática, es crucial comprender el fundamento conceptual detrás de cada línea de código. Un sistema distribuido se caracteriza por componerse de múltiples nodos independientes que se coordinan mediante paso de mensajes.

1.1 ¿Qué es una Llamada a Procedimiento Remoto (RPC)?

En un sistema centralizado clásico, una función se invoca mediante una *llamada a procedimiento local* (LPC), donde los argumentos se sitúan en la pila del hilo y la CPU salta a una dirección del mismo espacio virtual de memoria.

En un **Sistema Distribuido**, los procesos residen en espacios de memoria completamente aislados. El paradigma de **RPC (Remote Procedure Call)** busca ofrecer **transparencia de localización**: que la invocación a una función ejecutada en otra máquina o proceso se escriba y se perciba idéntica a una función local de C#.

> 💡 **El Principio de Transparencia de Red y sus Límites**
> Aunque RPC persigue la *transparencia sintáctica*, la teoría clásica (Falacias de la Computación Distribuida de Deutsch) demuestra que **nunca existe transparencia semántica absoluta**:
> • Una llamada local nunca falla por caída del cable de red o congestión de routers.
> • Una llamada RPC puede experimentar latencia indeterminada, fallos parciales o caídas del servidor remoto.
> Por ello, gRPC fuerza el uso de tipos asíncronos (`Task`), cancelaciones y el control explícito de excepciones remotas con `RpcException` .

1.2 Anatomía de una Llamada RPC: Stubs y Skeletons

Para ocultar la complejidad de los sockets TCP y la codificación de tramas binarias, el middleware genera automáticamente dos componentes fundamentales:

- **Stub (Lado Cliente):** Actúa como proxy o representante local del servicio remoto. Empaqueta los argumentos en un búfer binario estructurado (**Marshalling** o Serialización) y despacha la trama por la red.
- **Skeleton / Base Service (Lado Servidor):** Recibe el mensaje binario, extrae los objetos originales de C# (**Unmarshalling** o Deserialización), despacha la ejecución al método real en `RestauranteService` y empaqueta el resultado de retorno.

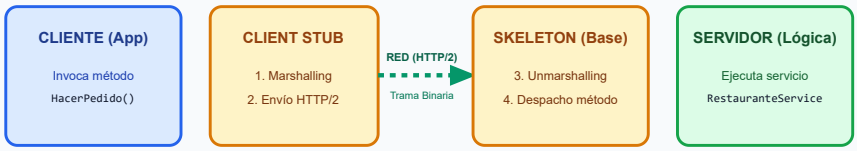


Figura 1.1: Flujo de invocación remota RPC mediante Stubs (Cliente) y Skeletons (Servidor).

1.3 ¿Por qué gRPC y Protocol Buffers en vez de REST y JSON?

Mientras que las arquitecturas web tradicionales emplean REST sobre HTTP/1.1 con texto JSON, las arquitecturas de microservicios distribuidos de alta concurrencia prefieren gRPC por motivos de eficiencia crítica:

Dimensión Técnica	Arquitectura REST con JSON	gRPC con Protocol Buffers
Formato del Mensaje	Texto plano estructurado (UTF-8). Ineficiente en tamaño y CPU.	**Binario compacto** mediante Varints y Tags de campo.
Contrato Formal	Opcional o tardío (OpenAPI/Swagger). Sujeto a desincronizaciones en runtime.	**Estricto y obligatorio** vía IDL (`.proto`). Comprobado en compilación.
Transporte de Red	HTTP/1.1 (o HTTP/2 no optimizado). Múltiples conexiones TCP.	**HTTP/2 puro** con multiplexación sobre un único socket TCP permanente.
Carga de CPU	Alta: El procesador parsea cadenas, comillas, llaves y convierte números a texto.	**Mínima:** Deserialización directa mediante desplazamientos binarios en RAM.
Streaming	Complejo (requiere WebSockets o SSE).	**Nativo de 4 tipos:** Unario, Client Streaming, Server Streaming y Bidireccional.

🔍 **Ejemplo Visual: Comparativa de Sobrecarga de Red (Payload Overhead)**

Para enviar un pedido con `id_pedido = 1` y `distancia = 5`:
• **En JSON (REST):** `{"id_pedido":1,"distancia":5}` ocupa **32 bytes** en texto ASCII.
• **En Protobuf:** Se codifica en solo **4 bytes** hexadecimales: `08 01 10 05` (Tag 1 = 1, Tag 2 = 5).
Resultado: ¡Un **87.5% de ahorro en ancho de banda** en cada llamada!

1.4 Concurrencia en un Servidor con Estado (Stateful)

En un servidor gRPC con estado en memoria, varios hilos atienden peticiones simultáneamente. Esto genera dos riesgos críticos:

1. **Condición de Carrera en Operaciones Compuestas (Check-Then-Act):** Si dos clientes hacen un pedido cuando la cola tiene 9 de 10 elementos, ambos hilos podrían comprobar la condición `Count < 10` a la vez y encolar simultáneamente, superando la capacidad máxima de 10 pedidos.
2. **Desperdicio de Hilos por Espera Activa:** Si el repartidor consultara la cola en un bucle continuo de sondeo (polling), saturaría la CPU de la máquina.

🎓 **Pregunta de Examen: ¿Por qué no usar 'Thread.Sleep' o un bucle 'while'?**

Un bucle `while (cola.IsEmpty) {{ Thread.Sleep(100); }}` retiene un hilo del pool de ASP.NET Core bloqueado inútilmente.
En su lugar, utilizamos un **semáforo asíncrono (** `SemaphoreSlim` **)**: el hilo del repartidor se libera mediante `await _readyOrdersSemaphore.WaitAsync()`, y Kestrel puede reutilizar ese hilo de CPU para atender a otros clientes. Cuando entra un pedido, `Release()` despierta al repartidor de manera reactiva e instantánea.

2. Arquitectura Global del Sistema

El sistema distribuido está estructurado en tres procesos independientes que se comunican en tiempo real mediante el puerto TCP 5000:

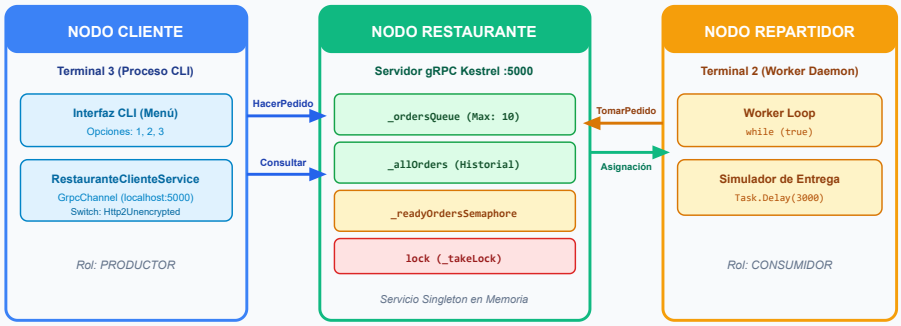


Figura 2.1: Arquitectura física de los tres procesos y canales de comunicación gRPC sobre HTTP/2.

2.1 El Patrón Productor-Consumidor con Búfer Acotado

El flujo algorítmico sigue las reglas canónicas de la computación concurrente:

1. **El Productor (Cliente):** Genera solicitudes mediante `HacerPedido`. Verifica que la cola no exceda la capacidad máxima (`MaxQueueCapacity = 10`). Si la cola está llena, el servidor rechaza la comanda devolviendo un error gRPC `StatusCode.ResourceExhausted`.
2. **El Búfer (Restaurante):** Mantiene la cola compartida `ConcurrentQueue<Pedido>` y un semáforo contador (`SemaphoreSlim`) inicializado a 0 permisos.
3. **El Consumidor (Repartidor):** Solicita pedidos llamando a `TomarPedidoParaRepartir`. Si no hay pedidos, queda a la espera en el semáforo. Cuando un cliente encola un pedido, ejecuta `_readyOrdersSemaphore.Release()`, despertando inmediatamente al repartidor.

3. Construcción del Proyecto Desde Cero con el CLI de .NET

Para levantar este proyecto desde un terminal en blanco (por ejemplo, en el examen de laboratorio), ejecuta los siguientes comandos exactos:

3.1 Paso 1: Creación de la Solución y los Proyectos

Terminal / Consola — Comandos CLI de Inicialización — BASH

```
# 1. Crear la carpeta raíz y acceder a ella
mkdir SistemaRestaurante
cd SistemaRestaurante

# 2. Crear la solución de .NET
dotnet new sln -n SistemaRestaurante

# 3. Crear el proyecto Web/Kestrel para el Servidor
dotnet new web -n Restaurante

# 4. Crear los dos proyectos de Consola para los clientes
dotnet new console -n Repartidor
dotnet new console -n Cliente

# 5. Agregar los tres proyectos a la solución
dotnet sln add Restaurante/Restaurante.csproj
dotnet sln add Repartidor/Repartidor.csproj
dotnet sln add Cliente/Cliente.csproj

# 6. Crear el directorio compartido para los contratos Protocol Buffers
mkdir Protos
```

3.2 Paso 2: Instalación de Dependencias NuGet de gRPC

Terminal / Consola — Comandos CLI para Instalar Paquetes NuGet — BASH

```
# 1. Paquete del Servidor (incluye Kestrel HTTP/2 y generador de server stubs):
dotnet add Restaurante package Grpc.AspNetCore

# 2. Paquetes del Repartidor (cliente HTTP/2, Protobuf runtime y compilador protoc):
dotnet add Repartidor package Grpc.Net.Client
dotnet add Repartidor package Google.Protobuf
dotnet add Repartidor package Grpc.Tools

# 3. Paquetes del Cliente:
dotnet add Cliente package Grpc.Net.Client
dotnet add Cliente package Google.Protobuf
dotnet add Cliente package Grpc.Tools
```

Estructura de Directorios Resultante

Verifica que la estructura de carpetas en tu workspace coincida con el siguiente esquema:

```
SistemaRestaurante/
├── SistemaRestaurante.slnx (o .sln)
├── Protos/
│   └── restaurante.proto
├── Restaurante/
│   ├── Restaurante.csproj
│   ├── Program.cs
│   ├── Pedido.cs
│   └── RestauranteService.cs
├── Repartidor/
│   ├── Repartidor.csproj
│   └── Program.cs
└── Cliente/
    ├── Cliente.csproj
    ├── Program.cs
    └── RestauranteClienteService.cs
```

4. El Contrato de Comunicación: Protos/restaurante.proto

El archivo `.proto` constituye la **Única Fuente de Verdad (Single Source of Truth)** del sistema distribuido.

Protos/restaurante.proto — Definición de Servicios y Mensajes Protobuf — PROTO

```
syntax = "proto3";
option csharp_namespace = "Restaurante";
import "google/protobuf/empty.proto";

service RestauranteService {
    rpc HacerPedido (PedidoRequest) returns (PedidoResponse);
    rpc ConsultarEstadoPedido (ConsultaRequest) returns (EstadoResponse);
    rpc TomarPedidoParaRepartir (google.protobuf.Empty) returns (PedidoResponse);
    rpc ConsultarPlatosPedido (ConsultaRequest) returns (PedidoResponse);
}

message PedidoRequest {
    repeated PlatoPedido platos = 1;
    int32 distancia = 2;
}

message PlatoPedido {
    string nombre = 1;
    int32 cantidad = 2;
}

message PedidoResponse {
    int32 id_pedido = 1;
    string mensaje = 2;
}

message ConsultaRequest {
    int32 id_pedido = 1;
}

message EstadoResponse {
    string estado = 1;
    string mensaje = 2;
    int32 tiempo_estimado = 3;
}
```

4.1 Desglose Exhaustivo Línea a Línea del Contrato

- `syntax = "proto3";` (Línea 1): Define la versión 3 de la sintaxis de Protocol Buffers. En `proto3` todos los campos son opcionales por defecto y no existe `required`, garantizando compatibilidad evolutiva sin roturas.
- `option csharp_namespace = "Restaurante";` (Línea 2): Especifica el espacio de nombres de C# en el que se ubicarán las clases generadas automáticamente.
- `import "google/protobuf/empty.proto";` (Línea 3): En gRPC, todo método RPC debe recibir obligatoriamente un mensaje y devolver otro. Google provee el tipo estándar `google.protobuf.Empty` para aquellos métodos sin argumentos de entrada.
- **Definición del Servicio `RestauranteService` (Líneas 5-10):** Declara 4 procedimientos remotos unarios:
  1. `HacerPedido (PedidoRequest) returns (PedidoResponse);`: El cliente envía la comanda con platos y distancia, y recibe el identificador del pedido.
  2. `ConsultarEstadoPedido (ConsultaRequest) returns (EstadoResponse);`: Permite consultar en qué fase se encuentra un pedido dado su ID.
  3. `TomarPedidoParaRepartir (google.protobuf.Empty) returns (PedidoResponse);`: Invocado por el repartidor para extraer el siguiente pedido listo.
  4. `ConsultarPlatosPedido (ConsultaRequest) returns (PedidoResponse);`: Devuelve la lista detallada de platos asociados a un pedido.
- **Colecciones con `repeated` (Línea 13):** La directiva `repeated PlatoPedido platos = 1;` genera en C# una colección de tipo `Google.Protobuf.Collections.RepeatedField<PlatoPedido>`.
- **Los Números de Campo (`= 1, = 2`):**

> 🎓 **Pregunta de Examen: ¿Qué significan los números '= 1' y '= 2' en los campos Protobuf?**
>
> ¡NO son valores por defecto! Son **etiquetas de campo binarias (Field Tags)**. En la trama binaria de la red, gRPC no transmite cadenas de texto como 'distancia', sino el tag numérico `2` en 1 byte. Por esta razón, **nunca se debe modificar el número de un campo en un sistema desplegado**, pues rompería la compatibilidad binaria.

5. Configuración de los Proyectos (.csproj)

La compilación de archivos Protobuf se gestiona mediante tareas de MSBuild integradas por el paquete `Grpc.Tools`.

5.1 Configuración del Servidor: Restaurante.csproj

Restaurante/Restaurante.csproj — Proyecto del Servidor gRPC — XML

```xml
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Grpc.AspNetCore" Version="2.84.0" />
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Server" />
  </ItemGroup>

</Project>
```

5.2 Configuración del Repartidor: Repartidor.csproj

Repartidor/Repartidor.csproj — Proyecto del Repartidor (Worker Client) — XML

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Google.Protobuf" Version="3.36.2" />
    <PackageReference Include="Grpc.Net.Client" Version="2.84.0" />
    <PackageReference Include="Grpc.Tools" Version="2.84.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Client" />
  </ItemGroup>

</Project>
```

5.3 Configuración del Cliente: Cliente.csproj

Cliente/Cliente.csproj — Proyecto del Cliente Interactivo — XML

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Google.Protobuf" Version="3.36.2" />
    <PackageReference Include="Grpc.Net.Client" Version="2.84.0" />
    <PackageReference Include="Grpc.Tools" Version="2.84.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Client" />
  </ItemGroup>

</Project>
```

> 💡 **El Porqué Técnico de 'GrpcServices=Server' frente a 'GrpcServices=Client'**
>
> Al compilar con `dotnet build`, el compilador `protoc` analiza el atributo `GrpcServices`:
> • `GrpcServices="Server"`: Genera la clase abstracta `RestauranteServiceBase`, que contiene los métodos virtuales que el servidor debe sobreescribir (`override`).
> • `GrpcServices="Client"`: Genera la clase concreta `RestauranteServiceClient` (el Stub), que implementa los métodos de red (`HacerPedidoAsync`, etc.).
> Si olvidáramos esta directiva, las clases correspondientes no se generarían y el código no compilaría.

6. Implementación del Servidor (Restaurante)

El servidor coordina todo el ciclo de vida de los pedidos, implementando el modelo de dominio, la configuración de Kestrel y la sincronización concurrente.

6.1 El Modelo de Dominio Interno: Restaurante/Pedido.cs

Restaurante/Pedido.cs — Clase Entidad de Pedido Interno — CSHARP

```csharp
using System.Collections.Generic;
using Restaurante;

namespace RestauranteApp
{
    public class Pedido
    {
        public int Id { get; set; }
        public List<PlatoPedido> Platos { get; set; } = new List<PlatoPedido>();
        public int Distancia { get; set; }
        public string Estado { get; set; } = "En preparacion";
    }
}
```

6.2 Punto de Entrada y Configuración de Kestrel: Restaurante/Program.cs

Restaurante/Program.cs — Configuración del Host Kestrel y Servicios — CSHARP

```csharp
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Server.Kestrel.Core;
using Microsoft.Extensions.DependencyInjection;
using System;
using RestauranteApp;

var builder = WebApplication.CreateBuilder(args);

// Configurar Kestrel para permitir HTTP/2 sin TLS
builder.WebHost.ConfigureKestrel(options =>
{
    // Configurar el servidor para escuchar en HTTP/2 sin TLS en el puerto 5000
    options.ListenLocalhost(5000, o => o.Protocols = HttpProtocols.Http2);
});

// Agregar servicios gRPC
builder.Services.AddGrpc();

// Registrar RestauranteService como Singleton para mantener el estado
builder.Services.AddSingleton<RestauranteService>();

var app = builder.Build();

// Registra el servicio gRPC para que pueda recibir y gestionar invocaciones remotas.
app.MapGrpcService<RestauranteService>();

// Configurar una página de inicio simple
app.MapGet("/", () => "Servidor gRPC del Restaurante ejecutándose...");

Console.WriteLine("Iniciando servidor gRPC del Restaurante...");
Console.WriteLine("Presiona Ctrl+C para salir");

app.Run();
```

Análisis de la Configuración de Kestrel y Dependencias

- **HTTP/2 sin TLS (Líneas 10-15):** Por defecto, gRPC requiere HTTP/2 mediante negociación ALPN sobre TLS. En desarrollo local sin certificados, configuramos Kestrel para escuchar en **HTTP/2 en texto claro (h2c)** en el puerto 5000: `options.ListenLocalhost(5000, o => o.Protocols = HttpProtocols.Http2);`.
- **El Singleton Esencial (Línea 21):** `builder.Services.AddSingleton<RestauranteService>();`

> 🎓 **Pregunta de Examen Clave: ¿Por qué Registrar RestauranteService como Singleton?**
> Por defecto, ASP.NET Core registra los servicios gRPC como **Transient** (se crea una instancia nueva por cada petición RPC y se destruye al terminar).
> Si `RestauranteService` fuera Transient, cada llamada a `HacerPedido` o `TomarPedidoParaRepartir` crearía una cola y un diccionario completamente vacíos. ¡Los pedidos se perderían de inmediato!
> Al registrarlo como **Singleton**, el contenedor de dependencias garantiza que existe una **única instancia en memoria** compartida por todos los clientes y repartidores durante toda la ejecución del servidor.

- **Mapeo del Servicio (Línea 26):** `app.MapGrpcService<RestauranteService>();` enruta el tráfico HTTP/2 gRPC entrante hacia los métodos de nuestra clase.

6.3 La Lógica de Negocio y Concurrencia: Restaurante/RestauranteService.cs

A continuación se presenta el código completo del servicio gRPC del servidor:

Restaurante/RestauranteService.cs — Servicio gRPC con Manejo de Concurrencia — CSHARP

```csharp
using Grpc.Core;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
using Google.Protobuf.WellKnownTypes;
using Restaurante;
using Microsoft.Extensions.Logging;

namespace RestauranteApp
{
    public class RestauranteService : Restaurante.RestauranteService.RestauranteServiceBase
    {
        private readonly ILogger<RestauranteService> _logger;
        private readonly ConcurrentQueue<Pedido> _ordersQueue;
        // Añadimos un diccionario para recordar todos los pedidos históricos,
        // ya que la cola los elimina cuando el repartidor los toma.
        private readonly ConcurrentDictionary<int, Pedido> _allOrders;
        private const int MaxQueueCapacity = 10;
        private readonly SemaphoreSlim _readyOrdersSemaphore;
        private readonly object _takeLock;
        private int _orderID;

        public RestauranteService(ILogger<RestauranteService> logger)
        {
            _logger = logger;
            _ordersQueue = new ConcurrentQueue<Pedido>();
            _allOrders = new ConcurrentDictionary<int, Pedido>();
            _readyOrdersSemaphore = new SemaphoreSlim(0);
            _takeLock = new object();
            _orderID = 1;
        }

        public override Task<PedidoResponse> HacerPedido(PedidoRequest request, ServerCallContext context)
        {
            lock (_takeLock) // lock the access to the queue to prevent race conditions
            {
                if (_ordersQueue.Count < MaxQueueCapacity) // checks if the queue is full
                {
                    var pedido = new Pedido // creates a new pedido with the given data
                    {
                        Id = _orderID,
                        Distancia = request.Distancia,
                        Platos = request.Platos.ToList(),
                        Estado = "En preparacion" // sets the initial state of the pedido to "En preparacion"
                    };

                    _ordersQueue.Enqueue(pedido); // adds the pedido to the queue
                    _allOrders.TryAdd(pedido.Id, pedido); // adds the pedido to the dictionary

                    _logger.LogInformation($"Recibido un nuevo pedido con ID {_orderID}");
                    _orderID++;

                    _readyOrdersSemaphore.Release(); // releases the semaphore to indicate that there is a new order

                    return Task.FromResult(new PedidoResponse // returns the response to the client
                    {
                        IdPedido = pedido.Id,
                        Mensaje = $"Pedido {pedido.Id} recibido, tiempo estimado: {pedido.Distancia * 5} mins"
                    });
                }
            }

            throw new RpcException(new Status(StatusCode.ResourceExhausted, "La cola está llena"));
        }

        public override Task<EstadoResponse> ConsultarEstadoPedido(ConsultaRequest request, ServerCallContext context)
        {
            // Buscamos en el historial global en lugar de la cola,
            // porque si el repartidor lo ha tomado, ya no está en la cola!
            if (_allOrders.TryGetValue(request.IdPedido, out var pedido)) // tries to get the pedido from the dictionary
            {
                return Task.FromResult(new EstadoResponse // returns the response to the client
                {
                    Estado = pedido.Estado,
                    Mensaje = $"Su pedido está {pedido.Estado}",
                    TiempoEstimado = pedido.Distancia * 5 // calculates the estimated time for the delivery
                });
            }
            else
            {
                throw new RpcException(new Status(StatusCode.NotFound, "El pedido no existe")); // throws an exception if the pedido is not found in the di
            }
        }

        public override async Task<PedidoResponse> TomarPedidoParaRepartir(Empty request, ServerCallContext context)
        {
            await _readyOrdersSemaphore.WaitAsync();
```

```
            lock (_takeLock)
            {
                if (_ordersQueue.TryDequeue(out var pedido))
                {
                    pedido.Estado = "En reparto";
                    return new PedidoResponse
                    {
                        IdPedido = pedido.Id,
                        Mensaje = "Pedido en reparto"
                    };
                }
            }

            throw new RpcException(new Status(StatusCode.NotFound, "No hay pedidos disponibles"));
        }

        public override Task<PedidoResponse> ConsultarPlatosPedido(ConsultaRequest request, ServerCallContext context)
        {
            if (_allOrders.TryGetValue(request.IdPedido, out var pedido))
            {
                var mensaje = string.Join(", ", pedido.Platos.Select(p => $"{p.Cantidad}x {p.Nombre}"));
                return Task.FromResult(new PedidoResponse {
                    IdPedido = pedido.Id,
                    Mensaje = mensaje
                });
            }
            else
            {
                throw new RpcException(new Status(StatusCode.NotFound, "El pedido no existe"));
            }
        }
    }
}
```

Desglose Exhaustivo de Concurrencia en RestauranteService

1. Estructuras de Datos y Sincronización (Líneas 15-22):

- `ConcurrentQueue<Pedido> _ordersQueue;` : Cola FIFO thread-safe que almacena los pedidos listos para ser recogidos por el repartidor.
- `ConcurrentDictionary<int, Pedido> _allOrders;` :

> 💡 **El Porqué del Diccionario Histórico ConcurrentDictionary**
> Cuando el repartidor invoca `TomarPedidoParaRepartir`, el pedido se **desencola** (se elimina de `_ordersQueue`). Si un cliente ejecuta posteriormente `ConsultarEstadoPedido(id)` y solo tuviéramos la cola, ¡el sistema diría que el pedido no existe!
> El `_allOrders` actúa como el **registro histórico persistente en memoria**, manteniendo el objeto accesible en tiempo constante O(1) para consultar su estado ('En reparto') en todo momento.

- `SemaphoreSlim _readyOrdersSemaphore = new SemaphoreSlim(0);` : Semáforo contador inicializado a **0 permisos**. Cuando no hay pedidos, cualquier repartidor que invoque `WaitAsync()` queda pausado asíncronamente. Al entrar un pedido, `Release()` suma un permiso y lo desbloquea.
- `object _takeLock = new object();` :

> ⚠️ **¿Por qué hace falta 'lock' si ConcurrentQueue ya es thread-safe?**
> Las colecciones concurrentes solo protegen operaciones atómicas aisladas (`Enqueue` o `TryDequeue`).
> Sin embargo, la secuencia: `if (_ordersQueue.Count < MaxQueueCapacity) {{ _ordersQueue.Enqueue( ... ); }}` es una operación compuesta (**Check-Then-Act**). Si dos peticiones leen `Count = 9` simultáneamente, ambas pasarán el `if` y la cola terminará con 11 elementos. El cerrojo `lock (_takeLock)` asegura que la comprobación y la inserción se ejecuten de forma atómica indivisible.

2. Análisis de los Métodos RPC:

- **`HacerPedido` (Líneas 34-65):** Adquiere el cerrojo `lock (_takeLock)`, comprueba si `_ordersQueue.Count < MaxQueueCapacity`. Si hay espacio, crea el pedido, lo añade a la cola y al diccionario histórico, incrementa el semáforo con `_readyOrdersSemaphore.Release()` y responde al cliente. Si la cola está saturada (10 pedidos), lanza una `RpcException(new Status(StatusCode.ResourceExhausted, "La cola está llena"))`.
- **`ConsultarEstadoPedido` (Líneas 67-84):** Busca en `_allOrders` mediante `TryGetValue`. Si existe, retorna su estado actual y el tiempo estimado (`distancia * 5` mins). Si no existe, lanza un error gRPC con `StatusCode.NotFound`.
- **`TomarPedidoParaRepartir` (Líneas 86-104):** Ejecuta `await _readyOrdersSemaphore.WaitAsync();`, suspendiéndose asíncronamente si no hay pedidos. Al recibir el permiso, adquiere el cerrojo, realiza `_ordersQueue.TryDequeue`, muta el estado a "En reparto" y devuelve los datos al repartidor.
- **`ConsultarPlatosPedido` (Líneas 106-120):** Recupera el pedido del diccionario y formatea la comanda de platos utilizando `string.Join`.

7. El Agente Consumidor (Repartidor)

El repartidor es un cliente gRPC que opera como un **Worker de fondo (Daemon)**. Consume pedidos continuamente en un bucle infinito y simula el trayecto físico de entrega.

Repartidor/Program.cs — Worker Loop Asíncrono del Repartidor — CSHARP

```csharp
using System;
using System.Net.Http;
using System.Threading.Tasks;
using Grpc.Net.Client;
using Google.Protobuf.WellKnownTypes;
using Restaurante;
using Grpc.Core;

namespace RepartidorApp
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var httpHandler = new HttpClientHandler
            {
                ServerCertificateCustomValidationCallback = HttpClientHandler.DangerousAcceptAnyServerCertificateValidator
            };

            var channel = GrpcChannel.ForAddress("http://localhost:5000", new GrpcChannelOptions { HttpHandler = httpHandler });
            var client = new RestauranteService.RestauranteServiceClient(channel);

            Console.WriteLine("Repartidor iniciado. Esperando pedidos para repartir...");

            while (true)
            {
                try
                {
                    var response = await client.TomarPedidoParaRepartirAsync(new Empty());
                    Console.WriteLine($"[Asignado] {response.Mensaje} (ID: {response.IdPedido})");

                    Console.WriteLine("Entregando...");
                    await Task.Delay(3000); // Simulate delivery time
                    Console.WriteLine($"[Entregado] Pedido ID: {response.IdPedido} entregado con exito.\n");
                }
                catch (RpcException ex)
                {
                    Console.WriteLine($"[Error] {ex.Status.Detail}");
                    await Task.Delay(2000); // Retry delay
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"[Excepcion] No se pudo conectar: {ex.Message}");
                    await Task.Delay(3000); // Retry delay
                }
            }
        }
    }
}
```

7.1 Análisis del Funcionamiento del Repartidor

- **Creación del Canal (Línea 20):** `GrpcChannel.ForAddress("http://localhost:5000", ...)` inicializa la conexión HTTP/2 hacia el restaurante.
- **El Bucle Infinito (Línea 25):** Ejecuta un ciclo continuo `while (true)` solicitando tareas de despacho.
- **Llamada Remota Asíncrona (Línea 29):** Invoca `await client.TomarPedidoParaRepartirAsync(new Empty())`. Gracias al semáforo en el servidor, esta llamada se queda en suspensión en el servidor sin consumir CPU hasta que entra un pedido.
- **Simulación de Reparto (Línea 33):** `await Task.Delay(3000);` suspende la tarea durante 3 segundos para simular el trayecto en moto/bici.
- **Tolerancia a Fallos y Reintentos (Líneas 36-45):** Si el servidor no está encendido o sufre una desconexión, captura `RpcException` y reintenta tras 2-3 segundos, impidiendo que el proceso muera.

8. La Aplicación Cliente (Cliente)

La aplicación cliente separa claramente la lógica de comunicación con gRPC de la interacción con el usuario en consola.

8.1 Servicio de Comunicación: Cliente/RestauranteClienteService.cs

Cliente/RestauranteClienteService.cs — Encapsulamiento del Stub Cliente gRPC — CSHARP

```csharp
using Grpc.Net.Client;
using Grpc.Core;
using System;
using System.Threading.Tasks;
using System.Collections.Generic;
using Restaurante;

namespace ClienteApp
{
    public class RestauranteClienteService
    {
        private GrpcChannel? _channel;
        private RestauranteService.RestauranteServiceClient? _client;

        public void ConnectAsync()
        {
            AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);

            _channel = GrpcChannel.ForAddress("http://localhost:5000");
            _client = new RestauranteService.RestauranteServiceClient(_channel);
        }

        public async Task HacerPedidoAsync(List<PlatoPedido> platos, int distancia)
        {
            var request = new PedidoRequest { Distancia = distancia };
            request.Platos.AddRange(platos);

            try
            {
                var response = await _client!.HacerPedidoAsync(request);
                Console.WriteLine($"[Exito] Pedido ID: {response.IdPedido} - {response.Mensaje}");
            }
            catch (RpcException ex)
            {
                Console.WriteLine($"[Error] Code: {ex.StatusCode}, Mensaje: {ex.Status.Detail}");
                throw;
            }
        }

        public async Task ConsultarEstadoPedidoAsync(int idPedido)
        {
            var request = new ConsultaRequest { IdPedido = idPedido };

            try
            {
                var response = await _client!.ConsultarEstadoPedidoAsync(request);
                Console.WriteLine($"[Estado] {response.Estado}");
                Console.WriteLine($"[Mensaje] {response.Mensaje}");
                Console.WriteLine($"[Tiempo Estimado] {response.TiempoEstimado} mins");
            }
            catch (RpcException ex)
            {
                Console.WriteLine($"[Error] No se pudo consultar: {ex.Status.Detail}");
            }
        }
    }
}
```

> ⚠️ **El Switch de Seguridad: Http2UnencryptedSupport**
>
> En la línea 17 encontramos: `AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);`
>
> **¿Por qué es indispensable?** Por defecto, el manejador de sockets de .NET rechaza negociar HTTP/2 sin cifrado TLS (https). Al conectarse a `http://localhost:5000` en texto claro, si este switch no estuviera activado en runtime, .NET lanzaría una excepción inmediata de tipo `InvalidOperationException: The HTTP/2 unencrypted support switch must be set`.

8.2 Interfaz de Usuario por Consola: Cliente/Program.cs

Cliente/Program.cs — Menú Interactivo de Usuario — CSHARP

```csharp
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Restaurante;
using ClienteApp;

namespace Cliente
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var clientService = new RestauranteClienteService();
            clientService.ConnectAsync();
            Console.WriteLine("Conectado al servidor gRPC.");

            while (true)
            {
                Console.WriteLine("\n=== Menú ===");
                Console.WriteLine("1. Hacer un pedido");
                Console.WriteLine("2. Consultar estado del pedido");
                Console.WriteLine("3. Salir");
                Console.Write("Opcion: ");

                var opcion = Console.ReadLine();

                // Keep asking for an option if it's invalid, without reprinting the whole menu
                while (string.IsNullOrWhiteSpace(opcion) || (opcion != "1" && opcion != "2" && opcion != "3"))
                {
                    if (!string.IsNullOrWhiteSpace(opcion))
                    {
                        Console.WriteLine("Opción no válida.");
                    }
                    Console.Write("Opcion: ");
                    opcion = Console.ReadLine();
                }

                if (opcion == "1")
                {
                    var platos = new List<PlatoPedido>();
                    while(true)
                    {
                        Console.Write("Nombre del plato (o vacio para terminar): ");
                        var nombre = Console.ReadLine();
                        if (string.IsNullOrWhiteSpace(nombre)) break;

                        Console.Write("Cantidad: ");
                        if (int.TryParse(Console.ReadLine(), out int cantidad))
                        {
                            platos.Add(new PlatoPedido { Nombre = nombre, Cantidad = cantidad });
                        }
                    }

                    if (platos.Count > 0)
                    {
                        Console.Write("Distancia de envio: ");
                        if (int.TryParse(Console.ReadLine(), out int distancia))
                        {
                            try
                            {
                                await clientService.HacerPedidoAsync(platos, distancia);
                            }
                            catch { }
                        }
                    }
                }
                else if (opcion == "2")
                {
                    Console.Write("ID del pedido: ");
                    if (int.TryParse(Console.ReadLine(), out int id))
                    {
                        await clientService.ConsultarEstadoPedidoAsync(id);
                    }
                }
                else if (opcion == "3")
                {
                    break;
                }
            }
        }
    }
}
```

Puntos Clave del Flujo de Usuario:

- **Validación Robusta (Líneas 28-36):** Si el usuario pulsa una opción inválida, el sistema muestra "Opción no válida" y vuelve a pedir la entrada sin reimprimir todo el menú repetidamente.
- **Composición Dinámica de Platos (Líneas 40-52):** Permite añadir platos en bucle hasta que se pulsa Enter en blanco.

- **Colección Protobuf (Líneas 24-26 en ClienteService):** La lista nativa `List<PlatoPedido>` se integra en el mensaje gRPC con `request.Platos.AddRange(platos)`.

9. Guía de Ejecución y Casos de Prueba Límite

Para verificar el comportamiento distribuido, debes ejecutar los tres proyectos simultáneamente en **tres ventanas de terminal independientes** desde la carpeta raíz.

9.1 Orden Secuencial de Puesta en Marcha

Paso 1: Iniciar el Servidor (Restaurante)

En la primera terminal, ejecuta:

```
dotnet run --project Restaurante
```

Salida esperada: `Iniciando servidor gRPC del Restaurante...`

Paso 2: Iniciar el Repartidor

En la segunda terminal, ejecuta:

```
dotnet run --project Repartidor
```

Salida esperada: `Repartidor iniciado. Esperando pedidos para repartir...`

Paso 3: Iniciar el Cliente

En la tercera terminal, ejecuta:

```
dotnet run --project Cliente
```

Salida esperada: `Conectado al servidor gRPC.` seguido del menú interactivo.

9.2 Escenarios de Demostración Práctica

Caso 1: Pedido Básico y Despacho Inmediato

1. En el **Cliente**, selecciona `1`. Introduce el plato: "*Hamburguesa Doble*", cantidad: *2*. Presiona Enter en blanco para terminar. Distancia: *4*.
2. **Respuesta en Cliente:** `[Exito] Pedido ID: 1 - Pedido 1 recibido, tiempo estimado: 20 mins`.
3. **Reacción en Servidor:** `info: Recibido un nuevo pedido con ID 1`.
4. **Reacción en Repartidor:**
   ```
   [Asignado] Pedido en reparto (ID: 1)
   Entregando...
   (espera 3 segundos)
   [Entregado] Pedido ID: 1 entregado con exito.
   ```
5. En el **Cliente**, durante los 3 segundos de reparto, pulsa opción `2` y consulta el ID `1`. Verás: `[Estado] En reparto`.

Caso 2: Saturación del Búfer Acotado (Cola Llena)

Para demostrar que el control de concurrencia y la capacidad máxima funcionan:

1. Detén temporalmente el **Repartidor** pulsando `Ctrl + C` en su terminal.
2. Desde el **Cliente**, realiza 10 pedidos consecutivos (IDs del 2 al 11). Todos se encolarán con éxito.
3. Intenta realizar el **pedido número 11**.
4. **Resultado:** El servidor rechaza la petición y el cliente muestra por pantalla:
   ```
   [Error] Code: ResourceExhausted, Mensaje: La cola está llena
   ```
5. Vuelve a iniciar el Repartidor (`dotnet run --project Repartidor`). Verás cómo procesa y vacía la cola acumulada uno por uno sin perder ningún pedido.

10. Banco de 10 Preguntas Típicas de Examen / Defensa Oral

A continuación se recopilan las 10 preguntas conceptuales más habituales con sus respuestas modelo para obtener la máxima calificación:

P1: ¿Por qué gRPC utiliza HTTP/2 en lugar de HTTP/1.1 y qué ventajas aporta a este sistema?

Respuesta: HTTP/1.1 sufre de bloqueo de cabeza de línea a nivel de aplicación (cada petición requiere una conexión TCP o esperar al turno anterior). HTTP/2 introduce **multiplexación mediante streams binarios bidireccionales** sobre una única conexión TCP permanente. Esto permite que múltiples llamadas (como consultar estado y hacer pedidos) viajen simultáneamente por el mismo socket sin bloquearse entre sí, además de comprimir cabeceras con HPACK.

P2: ¿Por qué es necesario registrar RestauranteService como Singleton en el contenedor de dependencias?

Respuesta: En ASP.NET Core, el ciclo de vida por defecto es *Scoped* o *Transient* (se instancia una clase por cada llamada y se destruye al finalizar). Nuestro servidor mantiene un **estado en memoria** (la cola de pedidos, el diccionario y el semáforo). Si no fuera Singleton, cada llamada RPC se ejecutaría sobre una instancia vacía, perdiendo el estado y haciendo imposible que el repartidor recoja los pedidos generados por los clientes.

P3: ¿Por qué se utiliza SemaphoreSlim en lugar de un bucle de comprobación periódica (polling) en el repartidor?

Respuesta: Un bucle de polling con `Thread.Sleep` incurre en *espera activa* o introduce latencia artificial innecesaria, además de malgastar CPU y saturar la red con llamadas vacías repetitivas. `SemaphoreSlim.WaitAsync()` proporciona un mecanismo de sincronización **reactivo y no bloqueante**: el hilo del repartidor cede el control al sistema operativo y solo es reactivado cuando el cliente produce un pedido mediante `Release()`.

P4: Si ConcurrentQueue ya es una estructura thread-safe, ¿por qué es imprescindible el bloque 'lock (_takeLock)'?

Respuesta: Las colecciones concurrentes solo garantizan atomicidad en operaciones individuales aisladas (`Enqueue` o `TryDequeue`). Sin embargo, verificar la capacidad máxima y luego encolar (`if (_ordersQueue.Count < MaxQueueCapacity) Enqueue()`) es una operación compuesta tipo *Check-Then-Act*. Dos hilos concurrentes podrían leer simultáneamente que hay 9 pedidos e insertar ambos a la vez, superando el límite de 10. El cerrojo `lock` asegura que la comprobación y la inserción se ejecuten como una única sección crítica atómica.

P5: ¿Por qué se mantienen simultáneamente una cola (_ordersQueue) y un diccionario (_allOrders)?

Respuesta: Tienen ciclos de vida y propósitos distintos. La cola implementa el búfer de despacho de pedidos pendientes (una vez que el repartidor lo toma, se elimina con `TryDequeue`). Si un cliente intentase consultar el estado de su pedido tras ser asignado, no lo encontraría en la cola. El diccionario actúa como el repositorio histórico persistente en memoria con acceso en tiempo constante O(1), permitiendo monitorizar el estado ("En reparto") hasta el final.

P6: ¿Qué función cumple la directiva GrpcServices en los archivos .csproj?

Respuesta: Indica a la herramienta `protoc` (a través de `Grpc.Tools`) qué clases de C# debe generar. Con `GrpcServices="Server"` genera la clase abstracta base con los métodos virtuales que el servidor debe sobreescribir. Con `GrpcServices="Client"` genera el Stub cliente con los métodos remotos asíncronos para invocar al servidor.

P7: ¿Por qué se requiere 'AppContext.SetSwitch("...Http2UnencryptedSupport", true)' en el cliente?

Respuesta: Por motivos de seguridad en producción, el cliente HTTP de .NET (`SocketsHttpHandler`) no permite por defecto negociar HTTP/2 sin TLS (texto claro / h2c). Dicho conmutador de compatibilidad autoriza explícitamente el uso de HTTP/2 en texto claro en local (`http://localhost:5000`).

P8: ¿Cómo gestiona gRPC los errores frente a una API REST tradicional?

Respuesta: Mientras que REST utiliza códigos de estado HTTP genéricos (400, 404, 500) y formatos de cuerpo arbitrarios, gRPC utiliza una especificación formal de 16 códigos de estado canónicos (`StatusCode`), tales como `NotFound` , `ResourceExhausted` , `Unavailable` , transmitidos mediante cabeceras HTTP/2 en los trailers (`grpc-status` y `grpc-message`), lanzando una `RpcException` fuertemente tipada en el cliente.

P9: ¿Qué semántica de comunicación RPC presenta este sistema?

Respuesta: gRPC por defecto ofrece una semántica **At-Most-Once (A lo sumo una vez)**: ante un fallo de red o timeout, la llamada no se reintenta automáticamente para evitar duplicar operaciones no idempotentes (como crear dos pedidos idénticos con doble cobro). Para lograr *At-Least-Once* el cliente debe implementar reintentos explícitos, y para *Exactly-Once* se necesitarían tokens de idempotencia en el servidor.

P10: ¿Qué tipos de streaming existen en gRPC y cómo se podría aplicar en esta práctica?

Respuesta: Existen 4 tipos: Unario (usado aquí), Server Streaming, Client Streaming y Bidireccional. En esta práctica, el método `ConsultarEstadoPedido` podría transformarse en un **Server Streaming RPC**: el cliente solicita el seguimiento de su pedido una sola vez, y el servidor mantiene el canal abierto enviando actualizaciones en tiempo real ("En preparación" -> "En reparto" -> "Entregado") a medida que ocurren los eventos, eliminando la necesidad de consultas periódicas manuales.

11. Chuleta de Depuración y Resolución de Errores

Durante el desarrollo y ejecución del laboratorio, es común encontrarse con ciertos errores típicos de configuración en entornos Windows y .NET:

Mensaje de Error / Síntoma	Causa Raíz	Solución Inmediata
`Failed to bind to address http://localhost:5000: address already in use`	Un proceso anterior del Restaurante quedó ejecutándose en segundo plano ocupando el puerto TCP 5000.	En PowerShell, mata el proceso ejecutando: `Stop-Process -Name "Restaurante" -Force` o busca el PID con: `Get-NetTCPConnection -LocalPort 5000`.
`The HTTP/2 unencrypted support switch must be set`	El cliente intenta conectar a `http://` (sin TLS) sin autorizar HTTP/2 en texto claro.	Añade antes de abrir el canal: `AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);`
`Status(StatusCode="Unavailable", Detail="Failed to connect to all addresses")`	El cliente o repartidor intentó comunicarse pero el servidor no está en ejecución o el puerto es erróneo.	Asegúrate de haber iniciado primero la terminal del servidor con `dotnet run --project Restaurante` y que esté escuchando en el puerto 5000.
`The type or namespace name 'RestauranteServiceBase' could not be found`	Falta compilar el archivo Protobuf o la directiva `GrpcServices="Server"` no está en el `.csproj`.	Verifica que `Restaurante.csproj` contenga `<Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Server" />` y ejecuta `dotnet build`.

12. Resumen de Competencias del Ingeniero Informático

Al completar este laboratorio, habrás interiorizado los conceptos clave exigidos en la ingeniería de software distribuida moderna:

- Diseño de interfaces formales e interoperables mediante **Protocol Buffers IDL**.
- Manejo de protocolos de transporte binarios y de alto rendimiento sobre **HTTP/2 (h2c)**.
- Control estricto de concurrencia multihilo con **secciones críticas atómicas, búferes acotados y semáforos asíncronos**.
- Gestión del ciclo de vida de componentes distribuidos mediante **Inyección de Dependencias (Singleton)** en .NET 10.
- Manejo canónico de excepciones remotas con **códigos de estado gRPC**.

Laboratorio de Sistemas Distribuidos · Práctica 2 · Diseñado con rigor académico para el Grado en Ingeniería Informática.